Master in informatics and Multimedia Engineering

Computer Vision and Mixed Reality

1st Project – Feature Based
Face Detection and Recognition for Augmented Reality

1. Goal

a. Develop a computer vision application for face detection and recognition that allows the inclusion of virtual elements aligned with real objects.

b. Familiarization with the OpenCV (**Open**-Source **C**omputer **V**ision) library, among others, for real-time application programming.

c. The application to be developed is Augmented Reality (AR), so it will have to run in real time, and the virtual objects will have to be aligned with the faces.

d. Students should implement the face detection (*a.*), recognition (*b.*) (feature extraction and classification) and augmentation (*c.*) pipeline (choose one of the presented methods).

2. Development

a. Face detection

i. Haar Cascade Classifier

OpenCV "CascadeClassifier" method is based on the algorithm of object detection using Haar feature-based cascade classifiers proposed by Paul Viola and Michael Jones in their paper, "Rapid Object Detection using a Boosted Cascade of Simple Features" in 2001.

ii. OpenCV Deep Neural Network module (dnn)

OpenCV has a deep neural network module and includes a pre-trained Convolution Neural Network (CNN) for face detection. This network is based on the ResNet architecture with a Single Shot Detector (SSD) for the detection phase.

iii. *dlib* toobox

Dlib is a toolkit containing machine learning algorithms and tools [5]. It provides a set of methods for face and facial landmark detection, including up to 68 facial feature point estimation. The face detector uses the Histogram of Oriented Gradients (HOG) feature combined with a linear classifier.

iv. *Mediapipe*

The MediaPipe Face Detector [6] task detects faces in images and videos. This task can locate faces and facial features within a frame. This task uses a machine learning (ML) model that works with single images or a continuous stream of images. The task outputs face locations, along with the following facial key points: left eye, right eye, nose tip, mouth, left eye tragion, and right eye tragion.

Use one of these algorithms to implement face detection in images collected in real time by a camera (see [1,2] for documentation and [3,4] as examples of implementation).

b. Face Recognition

i. Face database

Acquire a face database from the class (e.g., 10 face images per student, depending on the method). This set will be used in this project as the training and test sets. It may also be used as an online face database.

ii. Face normalization

For better recognition results, it is recommended to perform face normalization on the training set (rotation, scaling, cropping). An example is given by the MPEG-7 recommendation, where each face is represented by a monochrome image (256 levels) with 56 rows and 46 columns, with both eyes, right and left, perfectly aligned horizontally, and located in line 24, columns 16 and 31, respectively.

iii. Feature extraction

1. Principal Component Analysis (Eigenfaces | FisherFaces):
   Implement dimensionality reduction via PCA, transforming images into weight vectors in a face space. Use OpenCV's EigenFaceRecognizer or FisherFaceRecognizer from the FaceRecognizer class [7]. An example of how to implement the Eigenface and Fisherface methods can be found at [11].
2. A classical Approach (Local Binary Patterns Histograms - LBPH):
   Use the Local Binary Patterns algorithm to create local texture descriptors. Use OpenCV's LBPHFaceRecognizer from the FaceRecognizer class [7, 8].
3. Deep Learning Approach (Deep Embeddings):
   Use a pre-trained model (e.g., ResNet from the dlib or MediaPipe library) to extract a feature vector (embedding) of 128 or more dimensions [9].

iv. Classification

Any supervised classification technique can be used, such as KNN or SVM (scikit-learn [10] can be used), to compare the face to be classified against those in the database. The FaceRecognition class from OpenCV has its own classifier.

Consider an arbitrary number of classes involving students in the class (at least one), the teacher, and a rejection class if none of the faces are known.

Test the implemented classifier using the test set (different from the training set).

c. *Combine real and virtual objects.*

i. Virtual object normalization

Based on the face feature points extracted on phase a, normalize the virtual object intended to be superimposed on the identified face, for example, a hat, glasses, or another object of your choice. Use different objects for different faces/persons. Consider at least three different virtual objects: one for the student, one for the teacher, and one for others.

ii. Adding with mask

Add to the image, using a mask.

Bibliography

[1] - https://docs.opencv.org/master/d6/d0f/group__dnn.html#ga33d1b39b53a891e98a654fdeabba22eb

[2] - https://github.com/opencv/opencv/wiki/Deep-Learning-in-OpenCV

[3] - https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html

[4] - https://medium.com/data-science/face-detection-models-which-to-use-and-why-d263e82c302c

[5] - http://dlib.net/

[6] - https://developers.google.com/mediapipe/solutions/vision/face_detector

[7] - https://docs.opencv.org/3.4/da/d60/tutorial_face_main.html

[8] - https://medium.com/@vincents0511/face-detection-recognition-using-lbph-and-opencv-cea124d9846a

[9] - https://github.com/ageitgey/face_recognition

[10] - https://scikit-learn.org/

[11] - https://github.com/pmtmjorge/fisherface-recognizer-lab

Main *OpenCV* function

Goal	OpenCV function(s)
Read an image	*imread()*
Show an image	*imshow(), waitKey()*
Read images from a camera or a video file	*VideoCapture(); VideoCapture.get(); VideoCapture.read();*
Add images	*add(), addWeighted()*
Multiply images	*multiply()*
Absolute difference	*absDiff()*
Apply a blur or a median filter	*blur(), medianBlur()*
Resize an image	*resize()*
Scales, calculates absolute values, and converts the result to 8-bit.	*convertScaleAbs()*
Color conversions	*cvtColor()*
Contour extraction	*Canny()*
Image thresholding	*threshold()*
Calculates an affine matrix of 2D rotation.	*getRotationMatrix2D()*
Applies an affine transformation to an image	*warpAffine()*